SPRINT 03 · ANGULAR · LAB05

HttpClient, interceptores y service workers

Peticiones HTTP, CRUD, interceptores funcionales, PWA y funcionamiento sin conexión

Ejercicios de entrenamiento. Hay tres niveles y cada uno es un proyecto distinto e independiente, así que no necesitas el anterior para empezar. Los enunciados no incluyen la solución: usa «Cómo saber que está bien» para comprobar tu trabajo.

> Este LAB no tiene todavía Fastbook ni resumen en el repositorio. Los ejercicios se basan en el título del LAB y en la API actual de Angular: compáralos con el temario oficial cuando lo tengas.

Nivel	Proyecto	Practicas	Tiempo orientativo
Fácil	Lector de blog	`provideHttpClient`, peticiones GET y POST tipadas, `HttpParams` y estados de carga y error	1-1,5 h
Medio	Gestor de tareas	CRUD completo, interceptores funcionales, reintentos y `HttpContext`	2,5-3 h
Difícil	Recetario PWA	Service workers, `ngsw-config.json`, `SwUpdate`, modo sin conexión y sincronización	4-5 h

> Trabaja con Angular 22, componentes standalone, `@if` / `@for` e `inject()`. Crea cada proyecto por separado con `ng new <nombre> --style=scss --ssr=false`. Si prefieres decoradores (`@Input`, `@Output`) o funciones (`input()`, `output()`), ambas opciones valen. Angular 22 no usa Zone.js: guarda en un `signal` el estado que cambie fuera de un evento de plantilla (temporizadores, HTTP o suscripciones al router), o la vista no se actualizará.

Fácil

1-1,5 h

`lector-blog`

Lector de blog

Practicas: `provideHttpClient`, peticiones GET y POST tipadas, `HttpParams` y estados de carga y error

Vas a crear un lector de artículos con la API pública de pruebas `https://jsonplaceholder.typicode.com`.

Modelo

```
export interface Articulo { id: number; userId: number; title: string; body: string; }
export interface Comentario { id: number; postId: number; name: string; email: string; body: string; }
```

Requisitos

1. Registra `provideHttpClient(withFetch())` en `app.config.ts` y crea `ArticulosService`.
2. Lista los artículos de 10 en 10 con `HttpParams` (`_page` y `_limit`), con botones de anterior y siguiente.
3. Lee el total desde la cabecera `x-total-count` usando `observe: 'response'` y desactiva «Siguiente» en la última página.
4. En `/articulos/:id`, muestra el artículo y sus comentarios (`/posts/:id/comments`).
5. Añade un formulario de nuevo artículo con POST. La API responde con `id: 101` pero **no guarda el dato**: añádelo solo a la lista local.
6. Muestra estados de carga y de error. Para probar el error, usa temporalmente una URL incorrecta.

Cómo saber que está bien

- En la pestaña Red solo hay una petición por cambio de página.
- Un error muestra un mensaje claro y el botón «Reintentar» funciona.
- Todas las respuestas están tipadas: no hay `any` en el código.

Preguntas

- ¿Por qué un Observable de `HttpClient` no hace la petición hasta que alguien se suscribe? ¿Qué pasa si te suscribes dos veces?
- ¿Qué contiene un `HttpErrorResponse` cuando falla la red y cuando el servidor responde 404?

Medio

2,5-3 h

`gestor-tareas`

Gestor de tareas

Practicas: CRUD completo, interceptores funcionales, reintentos y `HttpContext`

Vas a crear un gestor de tareas contra una API local con JSON Server. La lógica común (token, errores, carga) estará en interceptores, no en los componentes.

API local

Crea `db.json` en la raíz y ejecuta `npx json-server@0.17.4 --watch db.json` (puerto 3000):

```
{
  "tareas": [
    { "id": 1, "titulo": "Repasar interceptores", "hecha": false, "prioridad": "alta" },
    { "id": 2, "titulo": "Probar JSON Server", "hecha": true, "prioridad": "baja" }
  ]
}
```

Requisitos

1. Crea `TareasService` con GET, POST, PATCH (marcar como hecha) y DELETE. Las respuestas actualizan un signal, sin recargar la página.
2. `authInterceptor`: añade `Authorization: Bearer token-demo` clonando la petición con `req.clone`. Solo se aplica a las URL de tu API.
3. `cargaInterceptor`: mantiene un contador de peticiones activas en `CargaService`. Usa `finalize` y muestra una barra si el contador es mayor que 0.
4. `reintentoInterceptor`: reintenta 2 veces, con `retry({ count: 2, delay: 1000 })`, **solo** en peticiones GET y con errores de red o 5xx.
5. `errorInterceptor`: traduce errores a mensajes («Sin conexión», «No encontrado», «Error del servidor») y los muestra con `NotificacionesService`. Vuelve a lanzar el error.
6. Registra todo con `withInterceptors([...])` y razona el orden elegido.
7. Crea el `HttpContextToken` `SIN_CARGA`: las actualizaciones silenciosas no activan la barra.
8. Al marcar una tarea, actualiza la vista antes de la respuesta y revierte el cambio si falla.

Cómo saber que está bien

- Todas las peticiones llevan el token en la pestaña Red (compruébalo en las cabeceras).
- Si apagas JSON Server, un GET se intenta 3 veces en total, pero un POST solo una.
- Con dos peticiones simultáneas, la barra no desaparece hasta que acaba la última.
- Una actualización fallida devuelve la tarea a su estado anterior.

Preguntas

- ¿Por qué hay que clonar la petición? ¿Por qué no se reintenta un POST automáticamente?
- ¿Qué orden siguen los interceptores al salir la petición y al volver la respuesta?

Difícil

4-5 h

`recetario-pwa`

Recetario PWA

Practicas: Service workers, `ngsw-config.json`, `SwUpdate`, modo sin conexión y sincronización

Un recetario debe seguir funcionando en la cocina aunque se corte internet. Usa JSON Server con recetas inventadas en `http://localhost:3000/recetas`.

Ejecución

> El service worker no funciona con `ng serve`. Compila y sírvelo así: `ng build` y después `npx http-server dist/recetario-pwa/browser -p 8080 -c-1`. Comprueba la ruta real de `dist` en tu proyecto. `localhost` cuenta como origen seguro, por lo que no necesitas HTTPS para practicar.

Requisitos

1. Ejecuta `ng add @angular/pwa` y revisa los archivos creados: `manifest.webmanifest`, `ngsw-config.json` y `provideServiceWorker`.
2. En `ngsw-config.json`, añade un `dataGroup` `api-recetas` con estrategia `freshness` y `timeout` de 3 s para la API.
3. Añade otro `dataGroup` con estrategia `performance` para las imágenes.
4. Crea `ConexionService` con un signal `enLinea`, basado en los eventos `online` y `offline` de `window`, y muestra un aviso sin conexión.
5. Permite marcar recetas como favoritas sin conexión: guarda la acción en una cola en `localStorage` o IndexedDB y actualiza la vista.
6. Al recuperar la conexión, envía la cola **en orden** con `concatMap` y borra cada acción solo tras confirmarse. Si un envío falla, detén la cola sin perder lo pendiente y reinténtalo más tarde.
7. Con `SwUpdate`, detecta `VERSION_READY` y ofrece «Actualizar» sin recargar a la fuerza. Comprueba antes `swUpdate.isEnabled`.

Cómo saber que está bien

- En DevTools, Aplicación > Service Workers, aparece el worker activo.
- En modo sin conexión, la aplicación carga al recargar y muestra las recetas ya visitadas.
- Tres favoritos marcados sin conexión llegan a JSON Server, en orden, al volver la red, sin duplicarse aunque recargues.
- Tras compilar una versión nueva, aparece el aviso y «Actualizar» carga los cambios.

Preguntas

- ¿Qué diferencia hay entre `freshness` y `performance`? ¿Cuál usarías para precios y cuál para imágenes?
- ¿Por qué un service worker no sustituye a una cola propia para enviar datos sin conexión?
- ¿Por qué `navigator.onLine` no garantiza que el servidor esté disponible?
- ¿Qué riesgo tiene guardar en caché respuestas con datos privados del usuario?